PHOENIXTEKK

Workplace Marketplace · Intune Solutions

Endpoint Privilege Management (EPM)

Implementation Solution Guide

Version 1.0.0 · 2026-06-14

workplace.phoenixtekk.com

Contents

Executive Summary — Endpoint Privilege Management (Microsoft Intune) 4

What you get . . . 4

The Phoenixtekk deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Endpoint Privilege Management (EPM) 5

What Endpoint Privilege Management is . . . 5

How it works . . . 5

The three elevation types . . . 5

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard Phoenixtekk deployment model . . . 6

2. Requirements Guide — Endpoint Privilege Management (EPM) 6

2.1 EPM licensing (Intune Suite or EPM add-on) — the gating requirement . . . 7

2.2 EPM client (elevation extension) reachability . . . 7

2.3 Supported Windows operating system . . . 7

2.4 Device management and identity . . . 7

2.5 Network access to EPM/Intune service endpoints . . . 7

2.6 Application inventory for elevation rules . . . 8

2.7 Local administrator strategy (the end goal) . . . 8

2.8 Administrative roles . . . 8

Requirements checklist (use before configuring) . . . 8

Global Requirements — Intune Solutions (category-level) 8

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 9

G5. Pilot security group . . . 9

G6. Supported Windows baseline . . . 9

G7. Network access to Microsoft cloud endpoints . . . 9

3. Implementation Guide — Endpoint Privilege Management (EPM) 10

Step 1 — Confirm the EPM license is assigned . . . 10

Step 2 — Create the pilot security group . . . 10

Step 3 — Create the Elevation settings (base) policy . . . 10

Step 4 — Assign the base policy to the pilot . . . 11

Step 5 — Gather the applications that need elevation . . . 11

Step 6 — Create an Elevation rules policy . . . 11
Step 7 — Assign the rules policy to the pilot . . . 12
Step 8 — Sync and test an elevation on a pilot device . . . 12
Step 9 — Review reporting . . . 12
Step 10 — Remove standing local-admin rights from pilot users . . . 12
Step 11 — Validate the pilot . . . 12
Step 12 — Expand to production . . . 13
5. Testing Guide — Endpoint Privilege Management (EPM) 13
Test 1 — The EPM client installed . . . 13
Test 2 — The base policy applied . . . 13
Test 3 — An approved app elevates with the correct type . . . 13
Test 4 — A non-approved app does NOT elevate . . . 13
Test 5 — User-confirmed justification is captured . . . 13
Test 6 — Support-approved flow (if used) . . . 14
Test 7 — Tenant-wide elevation reporting . . . 14
Test 8 — Standard-user posture (after Step 10) . . . 14
Test 9 — Compliance remains intact . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Endpoint Privilege Management (EPM) 14
A. License & availability issues . . . 15
B. Client (extension) install issues . . . 15
C. Policy not applying . . . 15
D. Elevation rule not matching (app won't elevate) . . . 15
E. App elevates when it shouldn't (too permissive) . . . 16
F. Support-approved requests stuck . . . 16
G. Reporting gaps . . . 16
H. Diagnostics quick reference . . . 16
I. When to escalate . . . 16
7. Best Practices — Endpoint Privilege Management (EPM) 17
8. Deliverables Summary — Endpoint Privilege Management (EPM) 17
Documentation . . . 17
PowerShell scripts . . . 18
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Endpoint Privilege Management (Microsoft Intune)

Phoenixtekk Workplace Marketplace · Intune Solutions

Standing local administrator rights are one of the largest unmanaged risks on a Windows fleet. A user who is a permanent local admin can install anything, disable security tooling, and turn a single phished click into a full device compromise — and most help desks hand out admin rights simply because a few applications and tasks demand elevation. **Microsoft Intune Endpoint Privilege Management (EPM)** breaks that trade-off: users run as **standard users every day**, and only the *specific* actions you approve are allowed to elevate — automatically, with a justification prompt, or via an explicit support approval — all centrally governed and fully audited.

This solution is a complete, **deploy-it-today** package for rolling out Endpoint Privilege Management on Intune-managed Windows devices. It is written for an administrator with little or no prior EPM experience: every requirement (including the **Intune Suite / EPM add-on license**) is verified before it is used, every configuration step is exact and in order, and every result is testable.

What you get

- A plain-language **Solution Overview** so you understand what EPM is, the three elevation types, and why running standard-user-by-default is the goal.
- A **Requirements Guide** that tells you exactly what must be true before you start — licensing, the EPM client extension, supported Windows — and how to fix anything that isn't.
- An **Implementation Guide** of precise, click-by-click Intune steps in the required order: enable EPM, deploy the base policy, build elevation rules, pilot, and expand.
- Five **PowerShell scripts** for device readiness discovery, deployment validation, tenant-wide elevation reporting (Microsoft Graph), safe remediation, and generating signed/hash-based elevation rules from a target application.
- A **Testing Guide** that proves elevations behave exactly as your rules intend — and that nothing else can elevate.
- A **Troubleshooting Guide** for the issues that actually occur in the field.

The Phoenixtekk deployment model

Every Phoenixtekk Intune solution follows the same shape: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** You will recognize this pattern in every solution you buy from us, so each new deployment feels familiar and low-risk.

Outcome

A measurable move toward **least privilege**: local admin rights removed from everyday users, the handful of legitimate elevations allowed in a controlled and audited way, and a complete record of every privileged action on the fleet — achievable by a single administrator in a controlled pilot within a day.

1. Solution Overview — Endpoint Privilege Management (EPM)

What Endpoint Privilege Management is

Endpoint Privilege Management (EPM) is a Microsoft Intune capability that lets users sign in and work as **standard users** while still being able to perform the **specific** tasks that normally require local administrator rights. Instead of granting a person permanent membership in the local **Administrators** group, you define **elevation rules** that allow a named application or task to run with administrative privileges — and nothing else does.

The result is **least privilege in practice**: the everyday account has no standing admin rights, the legitimate "I need to install/run this one thing" cases are still handled, and every elevation is governed by policy and recorded centrally.

How it works

1. You enable EPM for the tenant and deploy the **EPM client (the elevation extension)** to devices through Intune. The client is what intercepts elevation requests on the endpoint.
1. You deploy an **Elevation settings (base) policy** that turns EPM on for targeted devices and sets the default behavior and reporting scope.
1. You deploy one or more **Elevation rules policies**. Each rule identifies an application by **file attributes** (file name plus a **certificate** or **file hash**) and assigns an **elevation type**.
1. When a standard user tries to run an approved application elevated (e.g. right-click → **Run with elevated access**), the EPM client checks the rules and applies the matching elevation type — silently, after a prompt, or after support approval. Unmatched requests fall back to the normal Windows behavior (a UAC admin-credential prompt the standard user cannot satisfy).
1. Every elevation — allowed or denied — is reported back to Intune, giving you a tenant-wide audit trail.

Because rules are tied to a **certificate or hash**, an attacker cannot simply rename a malicious binary to a trusted application's name to gain elevation.

The three elevation types

- **Automatic** — the application elevates silently with no user interaction. Use sparingly, only for fully trusted, signed software, because the user is never prompted.
- **User confirmed** — the default and recommended type. The user must actively confirm (and, optionally, authenticate with Windows Hello / credentials and/or type a business justification) before the app elevates. This keeps a human in the loop and produces a clean justification record.
- **Support approved** — the elevation request is sent to Intune and must be **approved by an

administrator** before it runs. Use for high-risk or rarely-needed applications.

Benefits

- **Removes standing local admin** — shrinks the blast radius of phishing, malware, and credential theft.
- **Keeps users productive** — the genuine elevation cases still work, without a help-desk ticket each time.
- **Granular and tamper-resistant** — rules are scoped to specific signed/hashed applications.
- **Fully audited** — a central, tenant-wide record of who elevated what, when, and why.
- **Compliance** — directly supports least-privilege and Zero Trust mandates and audit requirements.

Limitations (be honest with buyers)

- EPM is **licensed** — it requires the **Microsoft Intune Suite** or the standalone **Endpoint Privilege Management add-on**. It is not included in base Intune Plan 1.
- It manages **Windows** devices (Windows 10/11) managed by Intune; it is not a macOS/Linux solution.
- Rules are as good as their definitions — an overly broad **Automatic** rule can re-introduce risk; prefer **User confirmed**.
- Removing users from the local Administrators group (the end goal) is a **change-management** exercise; plan communication and a support-approved safety valve during transition.
- EPM governs **elevation**, not application allow-listing — pair it with **WDAC/App Control** and **ASR** for defense in depth.

The standard Phoenixtekk deployment model

This solution uses **one** model so the process is repeatable across our catalog:

> ***Intune-managed, Microsoft Entra–joined Windows devices, with EPM enabled via a base policy and governed by certificate/hash-based elevation rules — User confirmed by default.***

Every step follows the Phoenixtekk pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** You start with users still holding admin rights, observe what they actually elevate (reporting), codify those into rules, then remove standing admin rights once the rules cover real-world usage.

2. Requirements Guide — Endpoint Privilege Management (EPM)

Verify **every** requirement below before you configure anything. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Do not assume any prerequisite already exists.

Category-wide prerequisites (a tenant, an Intune subscription, MDM enrollment, etc.) are documented once in `global-requirements-intune-solutions.md`*. This guide lists only what is specific to or especially important for EPM.*

2.1 EPM licensing (Intune Suite or EPM add-on) — the gating requirement

Description: Endpoint Privilege Management is a **paid add-on**. Each targeted user needs either the **Microsoft Intune Suite** license or the **standalone Endpoint Privilege Management add-on**. **Why required:** Without the license the EPM policy node is unavailable and policies will not apply — this is the single most common blocker. **How to verify:** Microsoft 365 admin center → **Billing** → **Licenses**; confirm **Intune Suite** or **Endpoint Privilege Management** appears and is assigned to your pilot users. In Intune, confirm **Endpoint security** → **Endpoint Privilege Management** is present and not gated behind a trial/upsell. **How to implement if missing:** Purchase the **Intune Suite** or the **EPM** add-on in **M365 admin center** → **Billing** → **Purchase services**, then assign the license to every targeted user (or a licensing group). Allow time for the assignment to propagate before deploying.

2.2 EPM client (elevation extension) reachability

Description: The EPM client-side extension must be able to install and call its cloud service. **Why required:** The extension is what intercepts and evaluates elevation requests on the device; the base policy triggers its installation. **How to verify:** After assigning the base policy, on a device check `C:\Program Files\Microsoft EPM Agent\` exists and the **Microsoft EPM Agent Service** (service name `EpmService`/**EPMAgent**) is running (`Get-Service` / `EPM-Discovery.ps1`). **How to implement if missing:** Ensure the base policy is assigned and the device has synced; confirm network access to EPM/Intune endpoints (see 2.5). The extension auto-installs once policy applies.

2.3 Supported Windows operating system

Description: Windows 10 (supported builds) or **Windows 11**, Enterprise/Pro, x64 or Arm64. **Why required:** EPM and its client extension require modern, supported Windows builds. **How to verify:** `winver` on the device, or Intune → device → **Hardware**. Confirm a currently serviced Windows 10/11 build with current cumulative updates. **How to implement if missing:** Patch via Windows Update / Update rings, or upgrade to Windows 11.

2.4 Device management and identity

Description: Devices are **Intune-managed** and **Microsoft Entra joined** (or hybrid joined). **Why required:** EPM policies are delivered through Intune to enrolled devices and scoped to Entra users/devices. **How to verify:** Intune → **Devices** → **All devices** (device listed, compliant); on the device `dsregcmd /status` shows `AzureAdJoined : YES`. **How to implement if missing:** Enroll the device in Intune and join it to Entra ID (OOBE, Autopilot, or **Settings** → **Accounts** → **Access work or school**).

2.5 Network access to EPM/Intune service endpoints

Description: Outbound HTTPS to Microsoft Intune and EPM service endpoints. **Why required:** The client downloads policy/rules and uploads elevation reports; support-approved requests round-trip to the service. **How to verify:** From a device, reach `https://login.microsoftonline.com` and the Intune service;

confirm reporting appears in **Endpoint security** → **Endpoint Privilege Management** → **Reports**. **How to implement if missing:** Allow Microsoft's published Intune/EPM URLs through proxy/firewall.

2.6 Application inventory for elevation rules

Description: Know which applications/tasks your users legitimately need to run elevated, and their **publisher certificate** or **file hash**. **Why required:** Each elevation rule identifies an app by file name plus certificate or hash; you cannot author accurate rules without this. **How to verify:** Use EPM **reporting in audit/observation** during the pilot, and/or run `New-EpmElevationRule.ps1` against a target executable to read its signer and SHA-256 hash. **How to implement if missing:** Run a pilot with reporting first to discover real elevations, then codify them. Prefer **certificate**-based rules (survive app updates) over **hash** where signed.

2.7 Local administrator strategy (the end goal)

Description: A plan to remove standing local-admin rights from everyday users once rules cover real usage. **Why required:** EPM's value is realized only when users are standard users; leaving everyone a local admin makes EPM cosmetic. **How to verify:** On a device, `net localgroup administrators` lists who is currently a local admin. **How to implement if missing:** Plan a phased removal (communications + a support-approved safety valve). Removing users from the local Administrators group is managed separately (e.g. an Intune **Account protection / Local users and groups** policy) — coordinate it with your EPM rollout.

2.8 Administrative roles

Description: The operator needs rights to create Endpoint security policies and approve support-approved elevations. **Why required:** Authoring EPM policies and approving elevations are privileged actions. **How to verify:** Entra → **Roles and administrators**; confirm **Intune Administrator** (or **Endpoint Security Manager**) for the operator. **How to implement if missing:** Assign **Intune Administrator**/**Endpoint Security Manager**; reserve Global Administrator for tenant-wide setup.

Requirements checklist (use before configuring)

- **Intune Suite or EPM add-on license** assigned to pilot users
- Devices Intune-enrolled and Microsoft Entra joined — `dsregcmd /status`
- Supported, patched Windows 10/11 (Enterprise/Pro)
- Network/proxy allows Microsoft Intune + EPM endpoints
- EPM client extension installs and **EPMAgent** service runs — `EPM-Discovery.ps1`
- Application inventory for elevation rules (cert/hash) gathered — `New-EpmElevationRule.ps1`
- Plan to remove standing local-admin rights after rules are proven
- Pilot Entra security group created

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** Phoenixtekk solution in the **Intune Solutions** category. They are published once at the category level on the marketplace so individual solution listings stay small. Confirm these are in place before deploying *any* Intune solution.

Each entry: **Description · Why required · How to verify · How to implement if missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads and **Tenant administration** → **Tenant status** shows an active subscription. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator** (or Global Administrator for setup). **Implement:** Assign **Intune Administrator** to the operator; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** (Intune → Devices → Enrollment) and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every Phoenixtekk solution is piloted to a small group before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists with intended members. **Implement:** Entra → **Groups** → **New group** → type **Security** → add pilot members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current cumulative updates. **Why:** Modern Intune features and security settings assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** (Intune → Devices → Windows → Update rings) or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach

`https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`.
Implement: Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The Phoenixtekk deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Endpoint Privilege Management (EPM)

Follow these steps **in order**. Each step must be complete before the next. All navigation is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) unless stated otherwise.

> ***Model:*** *Intune-managed, Microsoft Entra–joined Windows devices. We deploy an* ***Elevation settings (base) policy*** *plus* ***Elevation rules policies*** *to a pilot group, default elevation type* ***User confirmed****, and we* ***observe with reporting first*** *before removing local admin rights.* ***Approach:*** *Pilot → Validate → Expand, exactly like every Phoenixtekk solution.*

Step 1 — Confirm the EPM license is assigned

Before anything else, confirm your pilot users hold the **Intune Suite** or **Endpoint Privilege Management** license (Requirements §2.1). Without it, the policies below will not apply.

1. In **M365 admin center** → **Billing** → **Licenses**, confirm the license is owned and assigned to pilot users.
2. In Intune, open **Endpoint security** → **Endpoint Privilege Management** and confirm it is available (not showing only a purchase/trial prompt).

Step 2 — Create the pilot security group

1. Open `https://entra.microsoft.com` → **Groups** → **All groups** → **New group**.
2. **Group type:** Security. **Group name:** `EPM-Pilot`.
3. Add 3–5 pilot users (or devices) as **Members**.
4. Click **Create**.

Step 3 — Create the Elevation settings (base) policy

1. Go to **Endpoint security** → **Endpoint Privilege Management**.
2. Select the **Policies** tab → **+ Create Policy**.
3. **Platform:** Windows. **Profile: Elevation settings policy**. Click **Create**.
4. **Name:** `EPM - Base - Pilot`. Click **Next**.
5. On **Configuration settings**:

• **Endpoint Privilege Management: Enabled**.

• **Default elevation response: Deny all requests** *(recommended)* — only rules you author will elevate. (Choosing **Require user confirmation** as the default is broader; keep it denied for the pilot.)

• **Send elevation data for reporting: Yes**, and set **Reporting scope** to **All elevations** (so you observe everything during the pilot).

1. Click **Next**.

Step 4 — Assign the base policy to the pilot

1. On **Scope tags**, add tags if your org uses them, else click **Next**.
2. On **Assignments**, under **Included groups**, click **Add groups** → select `EPM-Pilot` → **Select** → **Next**.
3. On **Review + create**, confirm settings → **Create**.
4. On a pilot device, sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**) and confirm the **EPM client** installs (see Testing §). The base policy is what triggers the client install.

Step 5 — Gather the applications that need elevation

1. Let the pilot run for a short observation period with reporting on, **or** identify the target applications up front.
1. For each target executable, run `New-EpmElevationRule.ps1 -Path "C:\Path\app.exe"` to read the file name, **publisher certificate**, and **SHA-256 hash** you'll enter in the rule.
1. Prefer a **certificate** match for signed apps (rules survive app updates); use **hash** for unsigned, fixed binaries.

Step 6 — Create an Elevation rules policy

1. Go to **Endpoint security** → **Endpoint Privilege Management** → **Policies** → **+ Create Policy**.
2. **Platform:** Windows. **Profile: Elevation rules policy**. Click **Create**.
3. **Name:** `EPM - Rules - Pilot`. Click **Next**.
4. Click **+ Add** to create a rule, then set:

• **Rule name** and (optional) **Description**.

• **Elevation type: User confirmed** *(recommended default)*.

• Under **User confirmation**, require **Business justification** and/or **Windows authentication** for a stronger record (recommended).

• **File name:** the exact executable name (e.g. `setup.exe`).

• **File hash** *or* add a **Certificate** (upload the signer cert and choose **Certificate** as

the validation) — use the values from `New-EpmElevationRule.ps1`.

• Optionally set the **child process behavior** (deny by default unless the app legitimately spawns installers).

1. Add additional rules as needed, then click **Next**.

Step 7 — Assign the rules policy to the pilot

1. **Scope tags** → **Next**.
2. **Assignments** → **Included groups** → **Add groups** → `EPM-Pilot` → **Select** → **Next**.
3. **Review + create** → **Create**.

Step 8 — Sync and test an elevation on a pilot device

1. On a pilot device, force a sync (**Access work or school** → **Info** → **Sync**).
2. As a **standard user**, right-click an approved application → **Run with elevated access**.
3. Confirm the **User confirmed** prompt appears, supply justification/authentication, and the app runs elevated. Confirm a **non-approved** app does **not** elevate.

Step 9 — Review reporting

1. Go to **Endpoint security** → **Endpoint Privilege Management** → **Reports**.
2. Confirm elevations from the pilot appear (managed elevations, and unmanaged/denied attempts).
3. Use this data to refine rules — add the legitimate apps users tried to elevate.

Step 10 — Remove standing local-admin rights from pilot users

Once rules cover real usage, realize EPM's value by making pilot users standard users.

1. Communicate the change to pilot users (what changes, how to elevate approved apps, who to contact).
2. Remove the pilot users from the device **Administrators** group (e.g. via an Intune **Account protection → Local user group membership **policy targeting** `EPM-Pilot`**). Keep a** support-approved** safety valve for edge cases.

1. Re-validate with `EPM-Validation.ps1` and the Testing Guide.

Step 11 — Validate the pilot

1. Run `EPM-Validation.ps1` on a pilot device → confirms the client is healthy and the base policy applied.

1. Run `EPM-ElevationReport.ps1` → confirms tenant-wide elevation reporting for the pilot.
2. Complete the **Testing Guide** (Section 5) before expanding.

Step 12 — Expand to production

1. Create `EPM-Production`, add it to the **base** policy and the **rules** policy **Assignments**.
2. Roll out in waves (by department). Re-validate each wave with `EPM-Validation.ps1` / `EPM-ElevationReport.ps1` and tighten the default response to **Deny all** everywhere.
1. Keep rules certificate-based where possible and review the elevation report on a schedule.

Order of operations (summary)

`Confirm license → Pilot group → Base policy (enable + reporting) → Assign → Discover apps → Rules policy (User confirmed) → Assign → Test elevation → Review reports → Remove local admin → Validate → Expand`

5. Testing Guide — Endpoint Privilege Management (EPM)

Run these validations after the pilot is deployed (Implementation Steps 8–11) and again after each production wave. Each test states **what to do** and the **expected result**. Do not expand the rollout until every test passes for the pilot group.

Test 1 — The EPM client installed

Do: On a pilot device run `EPM-Discovery.ps1`, or check `Get-Service EPMAgent` and the folder `C:\Program Files\Microsoft EPM Agent\`. **Expected:** The **Microsoft EPM Agent** service is present and **Running**; the agent folder exists.

Test 2 — The base policy applied

Do: Intune admin center → **Endpoint security** → **Endpoint Privilege Management** → `EPM - Base - Pilot` → **Device/User check-in status. Expected:** Pilot devices/users show **Succeeded**. No **Error** or **Conflict**.

Test 3 — An approved app elevates with the correct type

Do: As a **standard user** on a pilot device, right-click an approved application → **Run with elevated access**. **Expected:** The **User confirmed** prompt appears (justification/authentication if configured); after confirming, the app runs **elevated**.

Test 4 — A non-approved app does NOT elevate

Do: As a standard user, attempt **Run with elevated access** on an application that has **no** rule. **Expected:** It is **denied** (no elevation), falling back to the standard Windows UAC admin-credential prompt the user cannot satisfy. The default response is **Deny all**.

Test 5 — User-confirmed justification is captured

Do: Perform an approved elevation that requires a business justification; enter text. **Expected:** The elevation proceeds and the justification is recorded in the EPM report (Test 7).

Test 6 — Support-approved flow (if used)

Do: Trigger an elevation for an app configured as **Support approved**; approve it in Intune → **Endpoint Privilege Management** → the elevation request. **Expected:** The request appears for approval; after admin approval the user can elevate; before approval it is blocked.

Test 7 — Tenant-wide elevation reporting

Do: Run `EPM-ElevationReport.ps1` (Microsoft Graph), and review **Endpoint security** → **Endpoint Privilege Management** → **Reports**. **Expected:** Pilot elevations appear (managed elevations, plus unmanaged/denied attempts) with user, file, and elevation type.

Test 8 — Standard-user posture (after Step 10)

Do: On a pilot device after local-admin removal, run `net localgroup administrators`. **Expected:** Pilot users are **no longer** members; day-to-day work continues, and only rule-approved apps elevate.

Test 9 — Compliance remains intact

Do: Intune → **Devices** → **Compliance**; review pilot devices after EPM deployment. **Expected:** Devices remain **Compliant**; no new compliance failures were introduced.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5, 7, 9 pass for **100%** of the pilot group (Test 6 if support-approved is used; Test 8 after local-admin removal)
Wave pass	Tests 1–4, 7, 9 pass for the wave before starting the next

Record results in your change log. If any test fails, use the **Troubleshooting Guide** (Section 6) and the `EPM-Remediation.ps1` script before proceeding.

6. Troubleshooting Guide — Endpoint Privilege Management (EPM)

Work top to bottom. Most field issues are a **missing EPM license**, a base policy that hasn't synced, or an elevation rule whose certificate/hash doesn't match the binary. Use `EPM-Discovery.ps1` / `EPM-Validation.ps1` to confirm state and `EPM-Remediation.ps1` for the safe fixes.

A. License & availability issues

Symptom	Likely cause	Remediation
EPM node shows only a **purchase/trial** prompt	No Intune Suite / EPM add-on owned	Purchase EPM (Requirements §2.1); confirm in **M365 admin** → **Billing**
Policy targets a user but **never applies**	User has **no EPM license**	Assign the **Intune Suite / EPM** license to the user (or a licensing group); allow propagation
Worked for some users, not others	License assigned to only part of the pilot	Confirm every pilot user is licensed; re-sync the device

B. Client (extension) install issues

Symptom	Cause	Remediation
`Get-Service EPMAgent` not found / agent folder missing	Base policy not applied yet	Confirm `EPM - Base - Pilot` is assigned and **Succeeded**; force sync with `EPM-Remediation.ps1`
Service present but **Stopped**	Transient / client unhealthy	Run `EPM-Remediation.ps1` (restarts the service); reboot; re-check with `EPM-Discovery.ps1`
Client never reports data	Network blocking EPM/Intune endpoints	Allow Microsoft Intune/EPM URLs through proxy/firewall (Requirements §2.5)

C. Policy not applying

Symptom	Cause	Remediation
Base/rules policy shows **Error** or **Conflict**	Two policies set conflicting EPM defaults	Keep **one** authoritative base policy; resolve the conflicting setting
Settings apply to nobody	Wrong assignment	Confirm `EPM-Pilot` is under **Included groups** and the user/device is a member
Changes don't take effect	Device not synced	`EPM-Remediation.ps1` forces an MDM sync; confirm in **Devices** → **All devices**

D. Elevation rule not matching (app won't elevate)

- Confirm the **file name** in the rule exactly matches the executable.
- Re-run `New-EpmElevationRule.ps1 -Path ""` and compare the **certificate**/**SHA-256

hash **to the rule — a single byte difference (e.g. after an app update) breaks a** hash **rule. Switch signed apps to a** certificate** rule so updates keep working.

- Confirm the rules policy is **assigned** and shows **Succeeded** for the device.
- Make sure the user used **Run with elevated access** (right-click), not a plain double-click.

E. App elevates when it shouldn't (too permissive)

- Find the offending rule — an over-broad **Automatic** rule or a wildcard match. Tighten it to a specific signer/hash and switch to **User confirmed**.
- Set the **default elevation response** to **Deny all requests** so only explicit rules elevate.
- Restrict **child process** behavior so an elevated installer can't spawn an admin shell.

F. Support-approved requests stuck

Symptom	Cause	Remediation
User's request never approved	No admin reviewing the queue	Approve in Intune → **Endpoint Privilege Management** → the elevation request; assign an approver role
Approval granted but app still blocked	Device hasn't synced the approval	Force sync (`EPM-Remediation.ps1`); retry the elevation

G. Reporting gaps

- No elevations in **Reports** → confirm the base policy has **Send elevation data for reporting:** Yes **and a** reporting scope** that includes the events you expect.
- `EPM-ElevationReport.ps1` returns nothing → confirm Graph permissions/consent and that reporting has had time to upload (allow for the reporting interval).

H. Diagnostics quick reference

```
Get-Service EPMAgent                        # EPM client service state
Test-Path "C:\Program Files\Microsoft EPM Agent"   # client installed?
dsregcmd /status                            # Entra join / device state
net localgroup administrators               # who is still a local admin
.\EPM-Discovery.ps1                         # full readiness/health summary
.\EPM-Remediation.ps1                       # force sync + restart client
```

Event logs: **Applications and Services Logs** → **Microsoft** → **Windows** → **AAD** (device registration) and the EPM client logs under the agent folder.

I. When to escalate

If a device passes `EPM-Discovery.ps1` (licensed, client running, Entra joined), the base/rules policies show **Succeeded**, the rule's certificate/hash matches, and an approved app still fails to elevate, collect the EPM client logs and the policy check-in status and open a Microsoft support case.

7. Best Practices — Endpoint Privilege Management (EPM)

• **Always pilot first.** Deploy to `EPM-Pilot` (3–5 users), validate fully, then expand in waves. Never enable EPM tenant-wide on day one.

• **Observe before you enforce.** Run the base policy with **reporting on** and a **Deny all** default first; let real elevations surface in the report, then codify the legitimate ones into rules.

• **Default to User confirmed.** Make **User confirmed** your standard elevation type and require a **business justification** (and Windows authentication for sensitive apps). Reserve **Automatic** for a small set of fully trusted, signed software.

• **Prefer certificate rules over hashes.** Certificate-based rules survive application updates; hash rules break on every new build. Use hashes only for unsigned, fixed binaries. `New-EpmElevationRule.ps1` gives you both.

• **Keep rules narrow.** Scope each rule to a specific file name + signer; control **child process** behavior so an elevated installer can't become a backdoor to a full admin shell.

• **Remove standing local admin — that's the point.** EPM only pays off once users are standard users. Plan the removal (communications + a support-approved safety valve) and execute it after rules are proven.

• **Use Support approved for high-risk apps.** Route rare or sensitive elevations through admin approval rather than a blanket rule.

• **Layer your defenses.** EPM governs *elevation*, not *what may run*. Pair it with **WDAC/App Control** and **Attack Surface Reduction** for defense in depth.

• **Review the elevation report on a schedule.** Watch for new denied attempts (rules to add) and unexpected automatic elevations (rules to tighten). `EPM-ElevationReport.ps1` exports this for review.

• **Communicate to users.** A short heads-up ("you're now a standard user — here's how to run approved apps elevated and who to contact") dramatically reduces help-desk tickets.

• **Document each wave.** Record which group, which date, which rules, and validation results in your change log for auditability.

• **Keep Windows and the EPM client current.** EPM depends on a supported, patched Windows build and a healthy client extension; enforce Update rings and monitor client health with `EPM-Discovery.ps1`.

8. Deliverables Summary — Endpoint Privilege Management (EPM)

Everything included in this Phoenixtekk solution package.

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What EPM is, the three elevation types, benefits, limitations, the deployment model
2. Requirements Guide	Every prerequisite (incl. the EPM license) with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered, click-by-click Intune steps
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`EPM-Discovery.ps1`	Device readiness: OS, Intune/Entra join, EPM client + service health, current local admins
`EPM-Validation.ps1`	Confirms the EPM client is healthy and the base policy applied for the device/user
`EPM-ElevationReport.ps1`	Tenant-wide EPM elevation reporting (Microsoft Graph) — managed, unmanaged, and denied
`EPM-Remediation.ps1`	Safe remediation: forces MDM sync and restarts the EPM client; `-ResetClient` re-registers it
`New-EpmElevationRule.ps1`	Reads a target executable's file name, signer certificate, and SHA-256 hash to author rules

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process that follows the Phoenixtekk model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Least privilege in practice: standing local-admin rights removed from everyday users, the legitimate elevations allowed in a controlled, audited way, and a tenant-wide record of every privileged action — deployable by a single administrator, validated before every expansion.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EPM-Discovery.ps1

Purpose: Reports whether the local device is ready for Microsoft Intune Endpoint Privilege Management: OS version, Microsoft Entra join / Intune enrollment state, the EPM client (elevation extension) install and service health, and which accounts currently hold local administrator rights. Run it before and during deployment to find devices that need remediation and to confirm the client landed.

How to run: Run on the device in a normal (non-elevated) PowerShell window: powershell -ExecutionPolicy Bypass -File .\EPM-Discovery.ps1 Optional: export a CSV row for fleet collation: .\EPM-Discovery.ps1 -CsvPath C:\Temp\epm-readiness.csv

EPM-ElevationReport.ps1

Purpose: Pulls Endpoint Privilege Management elevation reporting from Microsoft Graph so you can see, across the tenant, which users elevated which applications, the elevation type (automatic / user-confirmed / support-approved), and denied/unmanaged attempts. Use it to measure coverage during the pilot, find legitimate apps that need a rule, and produce an audit trail of privileged actions.

How to run: Requires the Microsoft.Graph PowerShell SDK and an account that can read EPM reports (e.g. Intune Administrator / Endpoint Security Manager). The script connects with the DeviceManagementConfiguration.Read.All scope. Install-Module Microsoft.Graph -Scope CurrentUser # one time powershell -ExecutionPolicy Bypass -File .\EPM-ElevationReport.ps1 -CsvPath C:\Temp\epm-elevations.csv

EPM-Remediation.ps1

Purpose: Fixes the most common EPM field issues without making risky changes: forces an Intune MDM policy sync (so the base/rules policies and the client install reach the device) and restarts the EPM client service if it is stopped. With -ResetClient it additionally re-registers the client to recover an unhealthy extension. Use after a policy change, when the client hasn't installed, or when EPM-Validation.ps1 reports NOT PROVISIONED.

How to run: Run on the device in an ELEVATED PowerShell window (service restart needs admin): powershell -ExecutionPolicy Bypass -File .\EPM-Remediation.ps1 To also re-register the client extension: .\EPM-Remediation.ps1 -ResetClient

EPM-Validation.ps1

Purpose: Validates, on a pilot/production device, that the EPM client is installed and running, that the device is Intune-managed and Entra joined, and that the EPM elevation context menu ("Run with elevated access") is available — i.e. the base policy has applied and the device can actually broker elevations. Run after Implementation Step 8 and after each wave.

How to run: Run on the device as the signed-in user in a normal PowerShell window: powershell -ExecutionPolicy Bypass -File .\EPM-Validation.ps1

New-EpmElevationRule.ps1

Purpose: Inspects a target executable (or MSI) and prints the exact values you need to create an Endpoint Privilege Management elevation rule in Intune: the file name, the SHA-256 file hash, the file/product version, and — if the file is signed — the publisher certificate details (subject, thumbprint). It recommends a CERTIFICATE rule for signed apps (survives updates) and a HASH rule for unsigned, fixed binaries. Run it during Implementation Step 5 for every app your users legitimately need to elevate.

How to run: powershell -ExecutionPolicy Bypass -File .\New-EpmElevationRule.ps1 -Path "C:\Path\to\app.exe" Optional: emit a JSON blob you can keep alongside your change log: .\New-EpmElevationRule.ps1 -Path "C:\Path\to\app.exe" -AsJson